SASEUL
ORIGIN

ArtiFriends

Contents

Definition of SASEUL Origin Nodes

1. Light Node
2. Validator
3. Supervisor
4. Arbiter

Scheme of SASEUL Origin Network

1. Network Type
2. Network Structure
3. Role Changing

Blocks

1. Block Configuration
2. Lifecycle and Data Transfer to Arbiter

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Consensus

1. Transaction Request Configuration
2. Transaction Validating
3. Round Validating

Handling Evil Nodes

1. Light Node
2. Validator
3. Supervisor
4. Arbiter

Appendix

1. Overall Scheme

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Definition of SASEUL Origin Nodes

1. Light Node

Light nodes are the most basic and simple nodes on the network.
They are directly related to blockchain wallet and practical services for the end users who do not actively contribute to the network. Each node has its unique private key and tracker and is capable of requesting transactions and viewing the transaction results.

2. Validator

Validators also sync all existing blocks on the network and are authorized to participate in the consensus process.
They are capable of receiving transaction requests and either accepting or denying them based on consensus to generate data in a block form. The fundamental consensus is derived from the SASEUL Consensus Algorithm. Validators communicate directly with light nodes, periodically hash generated blocks, and transfer those blocks to the arbiters.

3. Supervisor

Supervisors are nodes that sync all existing blocks on the network.
They differ from light nodes because they are meant to monitor and supervise the blockchain network. The supervisors check whether the network has generated a valid block and store the block if they are valid. They are capable of detecting erroneous hash values but they do not directly form transactions that confirm the error or punish the evil nodes.

4. Arbiter

Arbiters are storage nodes for all blocks created in the network.
They do not participate in the consensus process. The only responsibility they have is sending past data when a validator requests them for reference during validation.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Scheme of SASEUL Origin Network

1. Network Type

1. Public Network

In a public network, anyone can assign supevisors or validators as long as they abide by the network's rules. The one who initially creates the network can configure the rules and those rules are shared by all network participants.

2. Private Network

In a private network, supervisors and validators are assigned according to the rules configured by the administrators. The administrators have control over assigning supervisor and validator roles - they can either take on the role themselves or allow regular users to assume those roles.

2. Network Structure

SASEUL Origin network consists of arbiters, validators, supervisors and light nodes. All nodes have unique trackers that contain their addresses and are consistently synced among all participants in the network. In a regular end-user service setting, the four nodes serve the following roles:

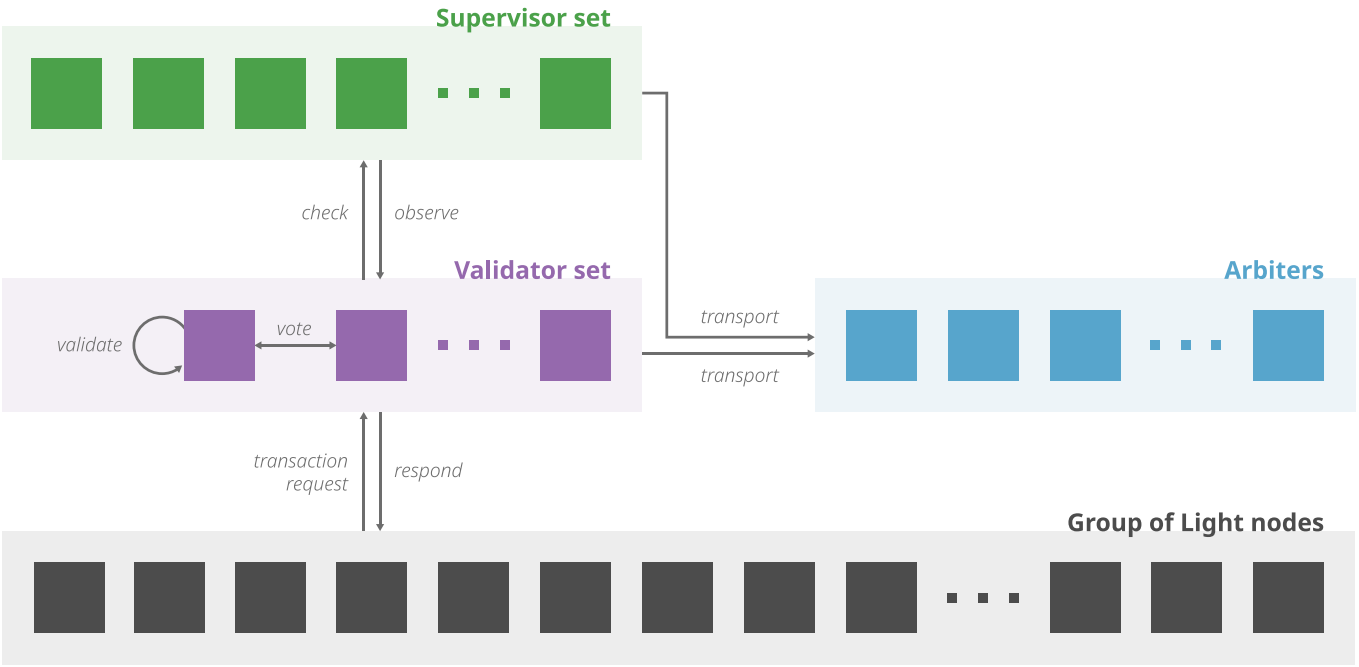


fig 1. Overall scheme of SASEUL Origin network. Please refer to the appendix for a more detailed scheme.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

1. Light Node

End-user programs are by default light nodes but they do not explicitly display that the program is a light node on the blockchain network. Light nodes send transactions requests to the validator and receive the execution result of those requests.

fig 2. Light nodes generate and send transaction requests.

2. Validator

In a public network, a node that created the network and commited the genesis block becomes the validator. In a private network, however, nodes selected by the administrators of that network assume the validator position. Validators accept or deny transaction requests from light nodes and periodically (or when data amount reaches a certain level) transfer those data to arbiter nodes. After the data transfer, validators hold the blocks' hash values only.

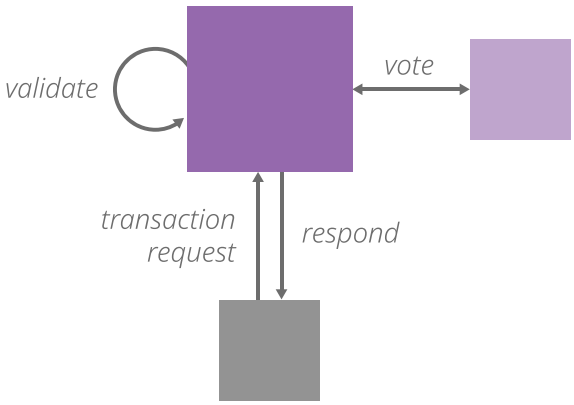


fig 3. Validators are directly involved in executing transactions.
They are capable of executing functions like approve, validate, vote, and respond.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

3. Supervisor

One can become a supervisor after completing the full chain sync and setting up supervisor configurations. Supervisors are capable of viewing all transactions and whether they were accepted or denied based on SASEUL consensus. They can also check if a correct hash was generated. However, supervisors are not authorized to report to the network or revert a transaction even if they discover erroneous or invalid transactions.

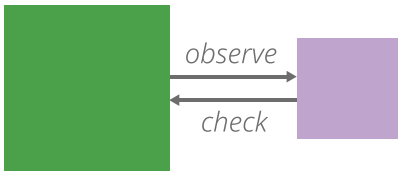


fig 4. Supervisors observe all consensus processes and determine their validity.
Validators use the determined result from supervisors as a reference.

> Q. Since supervisors are not authorized to execute any functions, are they similar to an observer?
>
> Although supervisors are not able to directly change the state of the network (i.e. fork), validators use the state of supervisors as a standard of accepting or denying transaction requests from heterogeneous blockchain groups. We are in the process of improving our algorithm and source code related to the supervisor role.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

4. Arbiter

Arbiters receive and store all block data from validators. The entire block chain is stored and if there are multiple arbiters, they all share the same data. In other words, the number of arbiters does not influence the performance of the network - it only ensures the safety of data.

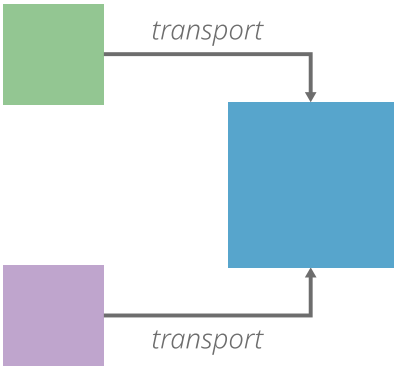


fig 5. Arbiters receive block data and store them at the end of each chain lifecycle.

3. Role Changing

1. Initial Role Granting

To start a network, there needs to be at least 3 validator nodes. A node that commits the genesis block and participates in the initial consensus becomes the first validator. When a network is first launched, there may or may not be light nodes or supervisors present. As the network grows, the number of supervisors and validators may increase.

2. Appending Supervisors and Validators

1) Supervisor

Anyone can apply to become a supervisor in the SASEUL blockchain network. Supervisor registration is managed by existing validators and the standard for registering a supervisor can be customized according to the structure of service and network. However, there are minimal requirements to assume the role of a supervisor - fully syncing with the past block data and depositing enough coin/token.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

2) Validator

In public network a supervisor can apply to become a validator. Validators will then reach consensus among themselves to decide whether to allow the supervisor to become a validator. Requirements for becoming a validator can be customized to meet the needs of that network. In a private network, nodes that fulfill the requirements set by the administrators can become validators.

If a new supervisor or validator is registered to the network, all nodes sync the registration information on their trackers.

3. Quitting the Network

SASEUL admin program can directly terminate a node on the network. In this process, the quitting node will still possess the deposit used to become a supervisor/validator - only the role will change to a light node.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Blocks

1. Block Configuration

A block contains the following data: transaction data, standard block time, and block hash. Definitions of each are as follows:

- **Transaction data:** transaction request and sender's signature with the approval status
- **Standard block time:** the standard time for every block generation
- **Block hash:** the hash value of the previous block

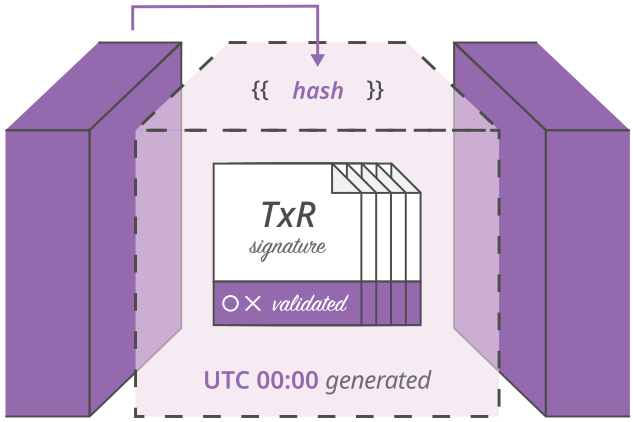


fig 6. Blocks contain previous block hash, validated transaction request, and block timestamp.

2. Lifecycle and Data Transfer to Arbiter

Lifecycle will be determined when creating the genesis block. Data memory size is generally used as the standard for determining the lifecycle.
When a chain lifecycle ends, the validators transfer all block data to the arbiter and hold only the blocks' hash values. Chain lifecycle is defined during the initial network configuration and is usally defined as time or blockchain length.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Consensus

The validators decide whether to accept or reject a transaction by reaching a consensus based on the SASEUL Consensus Algorithm. They independently determine the validity of each and every transaction and form a chunk of transactions to compare with other validators. In other words, rather than validating and syncing each and every transaction, a given range - specified by the network - of transactions will be validated altogether. The given range will be referred to as a **chunk** and the chunks are validated using their hash values.

1. Transaction Request Configuration

There are four essential elements included in a transaction requested by a light node. There may be additional data depending on the target service or industry.

- Transaction data

  Transaction data includes specifications and parameters needed for a transaction to be executed. For example, a transaction for sending coin would include sender address, recipient address, and amount.

- Timestamp

  The timestamp that represents when the transaction request has been made.

- Signature

  A signature generated using transaction data, timestamp, and private key. In SASEUL Origin, the transaction's hash value is signed directly.

- Public key

  A public key that can validate a signature generated by a private key. Since all nodes generate an address using a public key, one can trust the public key based on the address of other nodes.

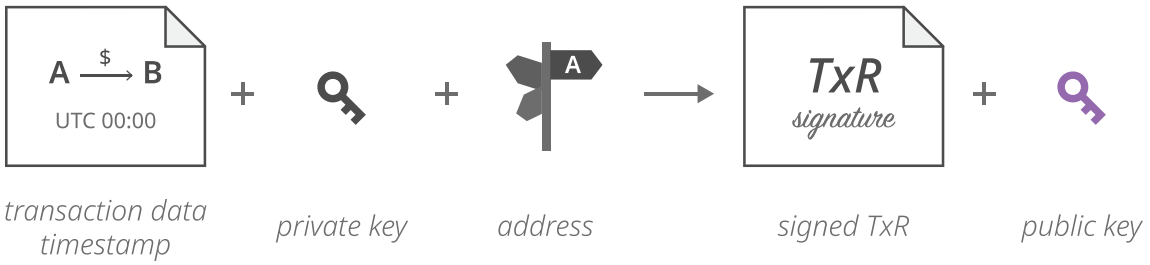


fig 7. Structure of a transaction request and how each request is generated

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

2. Transaction Validating

Each validator individually determines whether to accept or deny a transaction request and gathers the results in order to participate in the consensus. Validators decide whether to accept or deny based on two given standards:

- Is the sender really the one who requested the transaction?

  Signature, which is generated by a private key, is included in a transaction request. One can easily figure out the validity of a signature using a public key. And finally, one can trust the public key by comparing it to the address that requested the transaction.

- Is this transaction valid?

  Assuming that the sender is the one who requested the transaction, transaction itself must now be validated. For example, in the existing cryptocurrency networks, the system checks whether the sender has enough tokens to satisfy the request and whether the recipient address is valid.

3. Round Validating

1. Round sync

All validators store the length of their current operating chain and we named this value a **round**. In other words, round is the current block number at which a validator is operating within that chain lifecycle. In order to reach consensus among themselves, validators must check the rounds of other validators. Instead of broadcasting one's own round, SASEUL Origin requires a validator to request other validators' rounds.

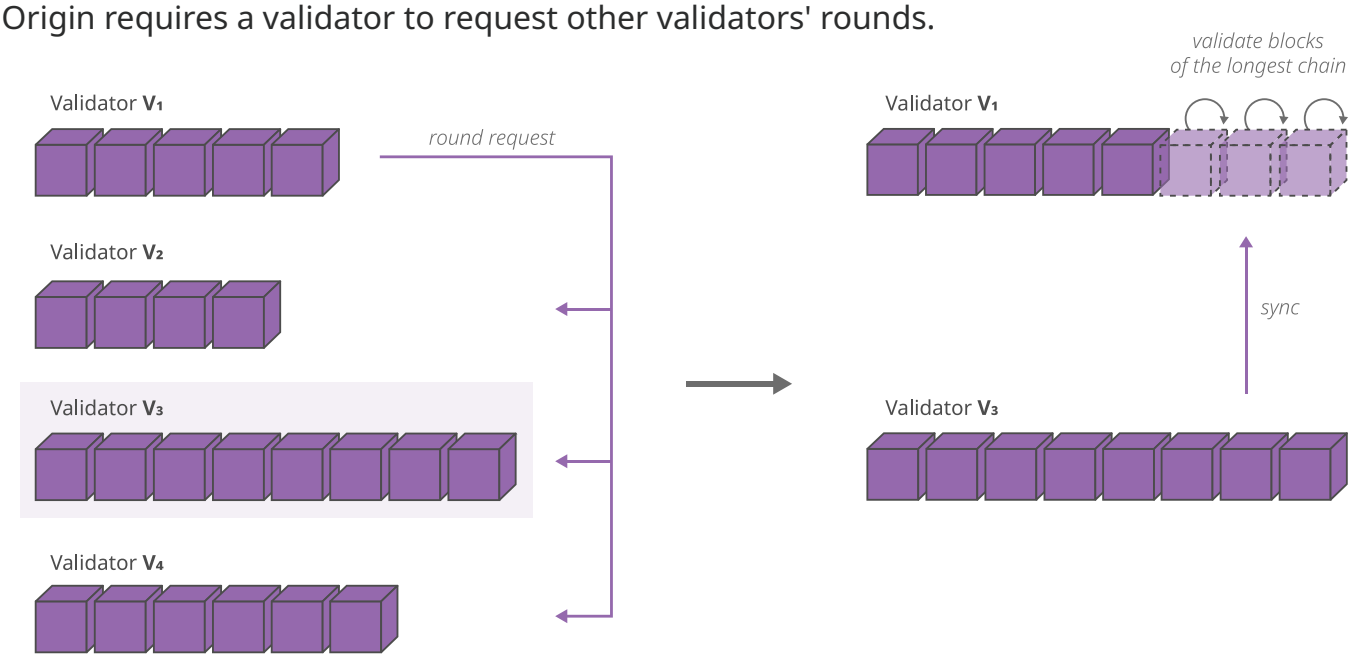


fig 8. Each validator requests rounds from other validators to sync with and validate blocks from rounds ahead.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

If there is a validator several rounds ahead, other validators who are several rounds behind would have to sync all rounds ahead and validate those extra rounds. If there are multiple nodes operating ahead, a lagging node would sync and validate data from the node operating at the highest round. If there are invalid data or transaction included, the recipient validator can remove that node from its tracker.
If there are no validators operating ahead, one can participate in a new round and generate blocks with other validators in the same round.

> Q. Why should rounds be requested, not broadcasted?
>
> Validators are constantly flooded with transaction requests while hash and requests are being transferred through the network. Since we cannot overlook the external factors of the network such as data speed and time difference, a perfect real-time synchornization is both theoretically and physically impossible. In such environment, if all validators broadcast their rounds as soon as their operations come to a close, there will be differences between expected and actual round values depending on the timestamp. The fact that validators cannot form a real-time synchronization means that validators do not have the ability to specify the time to form a consensus. In other words, any sort of interactions like voting cannot take place in such environment.

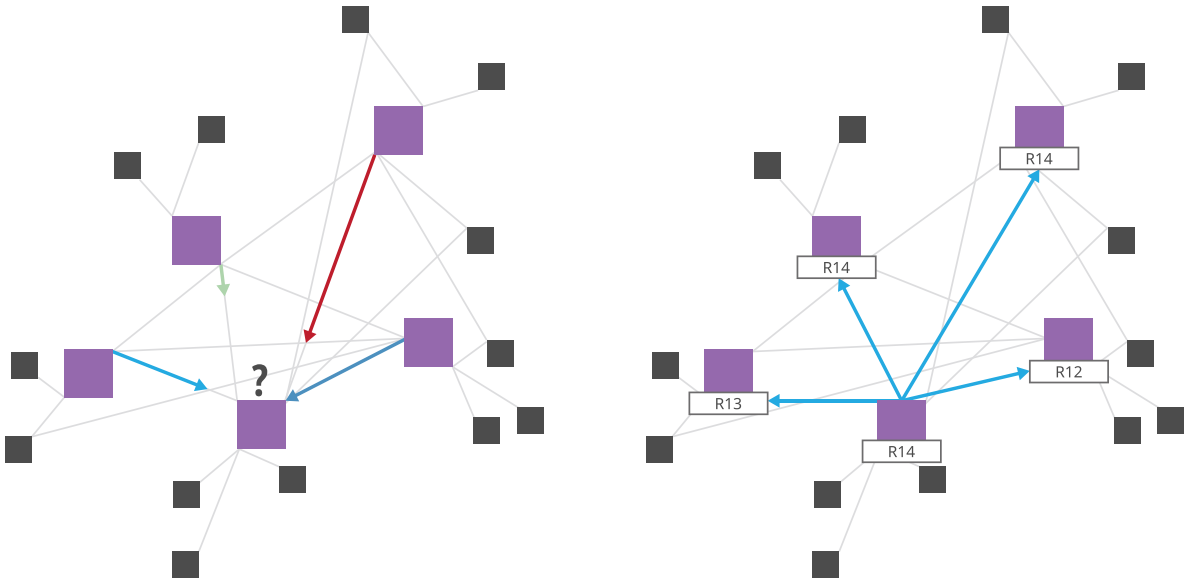


fig 9. Due to network synchronization issues, one cannot determine which round will be broadcasted and thus becomes impossible to determine when to start consensus. In other words, finality becomes meaningless. SASEUL consensus, on the other hand, requires validators to request round information from other validators.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

To get around this problem, SASEUL Origin allows validators to check the rounds of other validators and follow the round that they believe is valid. Validators in SASEUL Origin requests round information from other nodes instead of broadcasting them. In this way, if there is a node operating in rounds ahead, a validator could simply sync and validate the blocks ahead and determine whether to remain in the same network or not. On the contrary, if there are no validators operating in blocks ahead, one can simply participate in consensus with the validators in the same round.

In this process, validators are not putting pressure on or punishing other nodes to sort of the malicious ones. Instead, validators are allowed to cut out a node resulting in a different final value. In that case, even if the good nodes are the minority, the network will still survive.

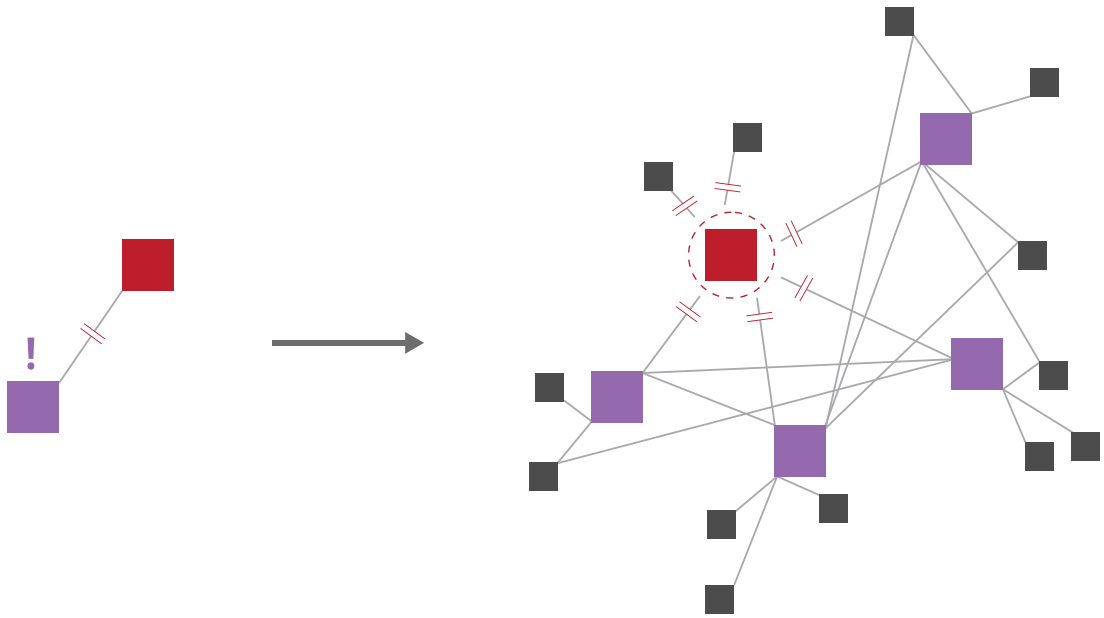


fig 10. When one node discovers an evil node (a node with different hash value), one can terminate connections with the evil node. Consequently, evil nodes will be isolated among themselves.

In existing consensus algorithms(i.e. PoS, PoW), consensus was dependent on the number of full nodes (or **stake**) in determining the validity. In other words, Byzantine General's Problem and the 51% attack was always the underlying stalemate. However, SASEUL Origin Consensus is unique in that it requires validators to request round information from other nodes instead of instead of synchronizing to form a single consensus. In essence, SASEUL Origin take a completely different logical perspective on forming a consensus. Therefore, even if the non-evil validator count falls below the 51% of the entire network, they can autonomously fork themselves out of the evil network and the light nodes are able to follow the non-evil validators by comparing their hash values.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Q. When do you request the round information?

The time at which a validator must request round information is determined according to the rules set out when the network was first created. At the assigned time, validators must request the round information from the previous time frame (**Trequest**).
Requesting round information before the previous time frame (**Trequest**) is impossible due to real-time synchronization. This **Trequest** value is determined after thorough considerations of factors like network state and distance. The time frame would then be calculated to assign enough time to request and receive round information.

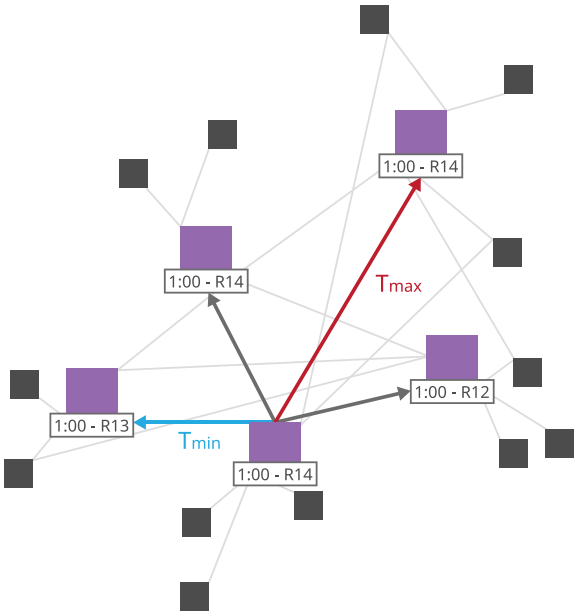


fig 11. Let T_{max} be the longest time to travel between two validators.
In that case, any value greater than $2T_{max}$ will be the longest response time in the network. Therefore, in order to get a reliable real-time response one must request for data from at least $2T_{max}$ back in time.

Here, the network distance refers to the time it takes for round information to reach the validator who sent the request. The longest distance (**Tmax**) will directly be involved in calculating **Trequest**. **Trequest** must be larger than **Tmax** doubled (round-trip).
To sum up, validators request round information during the time that has been agreed upon when the network was created. Hence, validators request round information from at least twice the longest network time-distance back in time to satisfy real-time sync.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

2. Block generate

If there are no validators operating ahead, one can generate the block for the current round. Each validator would generate a block and the consensus for that block would take place in the following round sync. In other words, if there is a malicious node including invalid data in a block, then that node would be removed from the non-evil nodes' trackers and thus will be forked from the network.

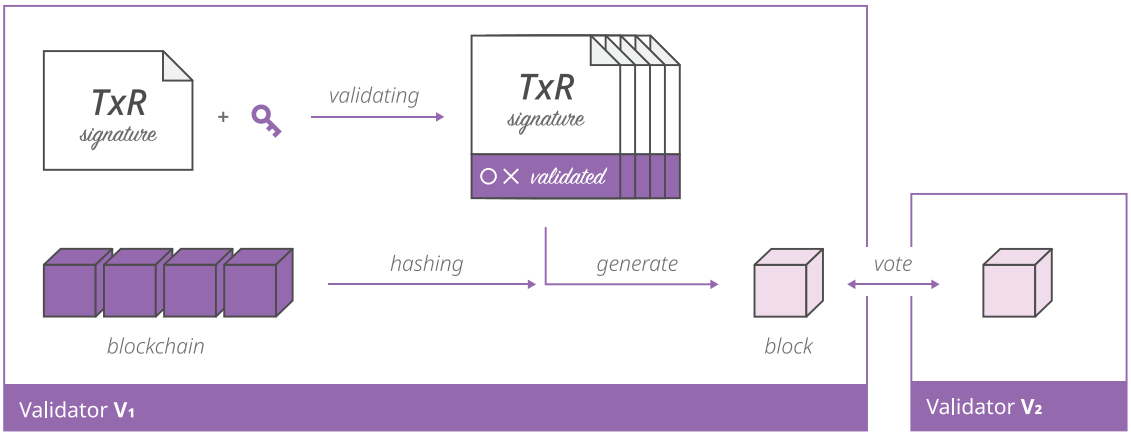


fig 12. If there are validators operating in the same round,
all validators in that round generate a block for that round and compare block hash among themselves.
If there are invalid block hashes, validators can terminate one another from the network.

3. Block commit

After the block has been generated, validators operating in the same round vote using the block hash in order to decide whether to commit the block or not. If the hash values are different, one will sync block with the validator that delivered the different hash value and check for any other differences. Based on the result, one can either update to re-commit or completely fork the network. However, no voting takes place after the block update - voting takes place only once during a single round.

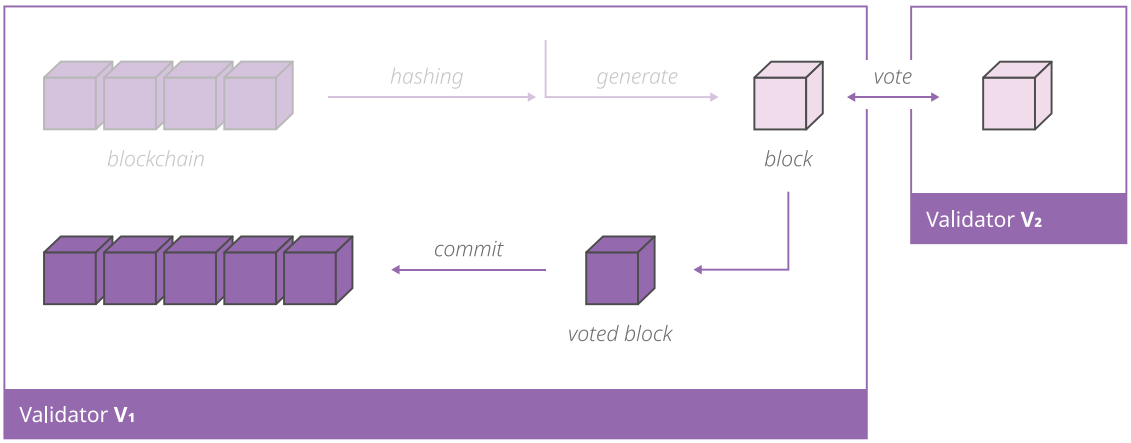


fig 13. A block that has reached consensus through voting is committed onto each individual chains.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Handling Evil Nodes

1. Light Node

Light nodes are only responsible for creating transaction requests and receiving data. In other words, evil behavior of a light node is limited to those related to transaction requests. Therefore, evil behaviors committed by light nodes are limited to two cases and can be prevented through the following:

- Continuously generating invalid transaction requests

  If the speed at which invalid transaction requests are made is not too fast, there won't be any request overload. SASEUL network can deny all invalid transactions.

- Overloading transaction requests

  Similar to DDoS attack, if light nodes are able to create excessive transaction requests, it will lead to a network overload. The way to solve this problem depends on the type of network, but most of the times it can be handled by setting a minimum interval between generating consecutive transaction requests. (i.e. 3 second interval)

2. Validator

Validators directly participate in the process of approving a transaction and therefore an evil behavior by a validator would be approving an erroneous transaction request. There is a variety of what is considered **erroneous transactions** but eventually, they are all the same when it comes down to resulting in different block hash.
If there is a validator with a block hash that is different from other validators due to an evil behavior, other validators can declare this validator to be evil. When declared evil, the evil validator will be removed from other validators' trackers and therefore the evil validator will be banished from the network.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

3. Supervisor

Supervisors only observe the network and does not directly participate in the network. Therefore, the only evil behavior that can be commited by a supervisor is delivering erroneous observations to the validators. In this case, validators can banish the supervisor from the network and prevent any further problems.

4. Arbiter

Arbiters receive and store block information and deliver them to the validators as a reference in case of an emergency. Therefore, possible evil behavior by an arbiter is storing erroneous information. Such problem can be solved through the following:

- Private network

  The administrator can own and manage a trustworthy arbiter, which creates a trust standard among arbiters, removing any potential risk.

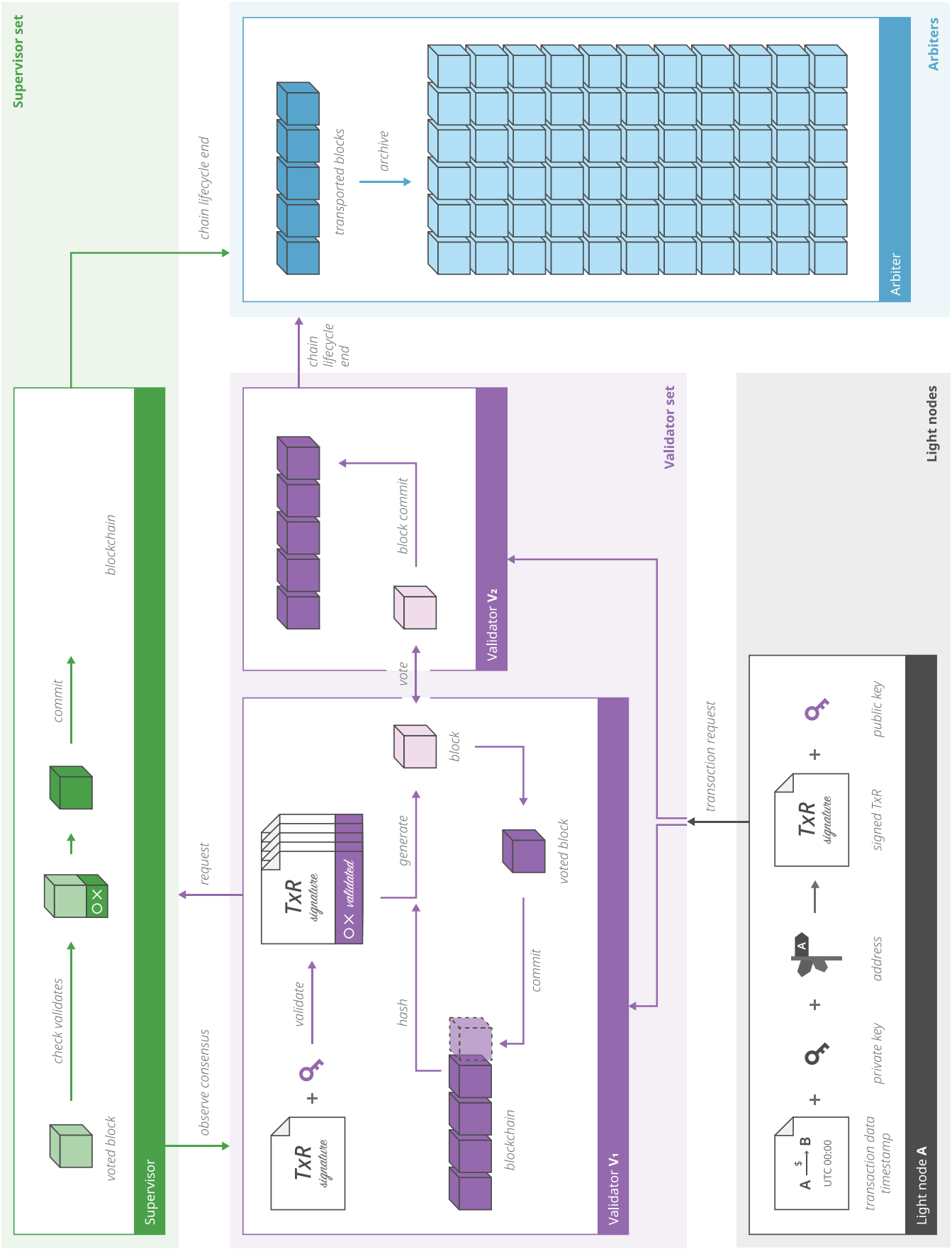
- Public network

  One can compare the block hashes of the validator and remaining arbiters. As a result, validators can banish the evil arbiter with a different hash value. Since all validators compare their hash values individually, as long as there is one non-evil arbiter, the network can be maintained. If all arbiters are evil, the validators can reach a consensus on a new arbiter.

THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.

Appendix I.

Overall Scheme



THIS FILE CONTAINS CONFIDENTIAL INFORMATION OR COPYRIGHT MATERIAL. IT IS NOT ALLOWED TO DISCLOSE, DISTRIBUTE, COPY, OR USE ANY OR ALL OF THE INFORMATION CONTAINED IN THIS DOCUMENT TO ANY OTHER THIRD PARTY.